

Slime' s Sky Travel

～Game Development Summary～

2025. 11. 22

日下 拓海

目次

1. 基本情報
2. ゲーム概要
3. 技術概要
4. 開発プロセス
5. まとめ

追記（2026. 6）

※本作品は私が初めて開発した作品で、ゲーム開発への理解度が不足していたため設計や可読性、保守性などが担保されていません。例えば public 変数が多く、setter、getter を使わず直接変更してしまっていたり、適切な粒度で関数分割を行っていないため同じ処理を繰り返し書いてしまったりしています。その反省を生かし、本作の精神的続編として Slime' sSpaceTravel という作品を開発しました。私の最新の技術力や設計力を評価いただく場合はぜひ本作品と続編を比較していただきたいです。

SlimesSpaceTravel ページ <https://github.com/kusaka0914/SlimesSpaceTravel/>

1. 基本情報

ゲーム基本情報

ゲームタイトル：Slime' s Sky Travel

コアコンセプト：ゲーム世界の速度を切り替えながら進む空中アクションゲーム

対応プラットフォーム : PC (macOS、Windows)

使用言語 : C++

制作期間 : 2025.8 から約1ヶ月

所要時間数の概数 : 200 時間程度

動作環境 : Windows10、macOS

制作人数 : 1 人

ゲーム開発における役割 : 全工程を担当しました

ゲームアピールポイント (3 点)

- ・ゲーム世界の速度を切り替えることで自分に合った速度感でプレイ可能です。速度を速くすると、クリアタイムを縮めることができる一方で操作が難しくなったり、ギミックが速くなったりし、ゲーム性にメリハリが生まれます。
- ・オンラインランキングに対応したタイムアタックやシークレットスターといったこだわり抜いたゲームモードに加え、タイムアタックモード中にリプレイ保存し、クリア後にプレイを再現することで自分のプレイを見て学習できるようにしました。また、このリプレイ機能はオンライン対応しており、他のプレイヤーのリプレイも閲覧できるようにしています。
- ・イージーモードによる救済を手厚くすることでアクションが苦手な方にも遊んでいただけるようにしています。一方、クリア後には高難易度モードを導入しているため、上級者の方にも楽しんでいただけるように設計しています。

開発者情報

開発者名 : 日下 拓海

所属 : 弘前大学理工学部電子情報工学科

連絡先 : takumi.090414528@gmail.com

プレイ動画/紹介動画

https://youtu.be/WSxkKcqc_a4?si=Ge0HI0TH5Cim0S0S

本作のプレイ映像と、基本ルール、特徴、実装したモード等に関する紹介をまとめ

た 3 分 40 秒の動画になります。



2. ゲーム概要

基本ルール

プレイヤーはゲーム世界の速度を切り替えながら、各ステージに 3 つ存在しているアイテムを全て集めてゴールを目指します。総ライフ数が 6 であり、落下するごとに 1 つのライフが失われます。全てのライフがなくなった時点でゲームオーバーになります。

コアゲームループ

ステージ選択フィールド上を移動してステージを解放/開始→ゲーム速度の設定→ゲームスタート→空中アクション→アイテムの位置を確認し取得→ゴール/星の獲得→フィールドに戻り次ステージに進む

操作方法

W/A/S/D : 移動

T : ゲーム速度変更 (1x, 2x, 3x)

ESC : セーブ/ゲーム終了

ゲームモード一覧

・ ノーマルモード

通常のゲームモードになります。できるだけ多くの星を獲得するために最速でのクリアを目指してあらゆるギミックを乗り越えていきます。

・ タイムアタックモード

ステージのクリアタイムを測定/記録できるモードになります。最速クリアタイムの更新を目的に行います。このモードにはリプレイ機能がついており、自分のプレイを見返して振り返ることが可能です。また、リプレイ中には一時停止、巻き戻し、早送り、リプレイ速度の変更を行うことができるため、自分のプレイ/ルートどりをじっくりと観察/研究することが可能になっています。また、測定したタイム/リプレイはサーバーに送信され、オンラインランキングに掲載されます。ゲーム内掲示板からタイムが早い人のプレイを閲覧することができるため、プレイの研究になります。(2026.6 現在はサーバーを停止しています。)

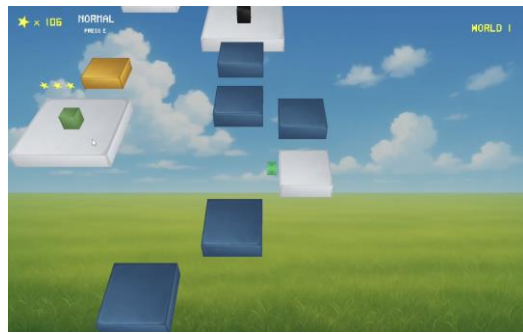
・ シークレットスターモード

ゲームクリア後に解放される、上級者向けのコンテンツになります。世界の速度が常に3倍になる Max Speed Star、世界が暗闇に包まれ、自分の足元しか見ることができなくなる Shadow Star、スライムの目線 (FPS 視点) になってステージを攻略する Immersive Star の3種類のシークレットスターが存在しています。

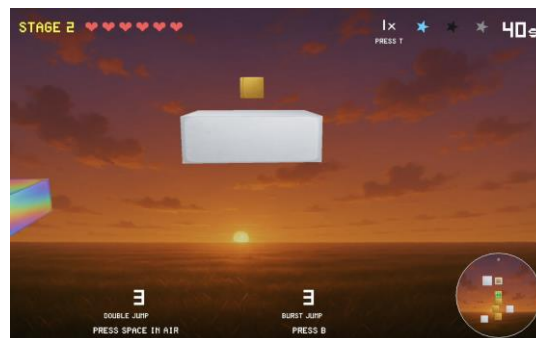
実際のゲーム画像



基本のノーマルモード



ギミックが体験できるステージ選択フィールド



Immersive Star モード

プレイ時間の目安（プレイヤーの習熟により異なります）

1 ステージ：1 分～5 分

想定クリア時間：10 分～30 分

想定プレイヤー像

- ・アクションゲームが苦手な方（イージーモード対応）
- ・空中アクション/アトラクション系のゲームを好んでいる方
- ・短い時間でゲームクリアまで楽しみたい方

※詳しいルールや各モードの実際のプレイは、紹介動画で解説しています。

3. 技術概要

使用技術

エンジン/フレームワーク :

- ・ 自作エンジン (GLFW + OpenGL)

言語 :

- ・ C++

主なライブラリ :

- ・ GLFW (ウィンドウ管理・入力処理)
- ・ OpenGL (グラフィックス)
- ・ GLM (数学ライブラリ)
- ・ SDL2_mixer (オーディオ)
- ・ nlohmann/json (JSON 処理)
- ・ stb_image (画像処理)

開発環境 :

- ・ クロスプラットフォーム対応 (Windows / macOS)

ビルド・ツール類 :

- ・ CMake (ビルドシステム)
- ・ Git / GitHub

主なシステム・モジュール

・ **GameLoop** : メインゲームループを管理し、毎フレームの更新・描画順序を制御します。タイトル画面、Ready 画面、ステージクリア画面などの画面遷移も統合管理します。

・ **StageManager** : ステージの読み込み・解放・進行状況を管理します。JSON ファイルからステージデータを読み込み、ステージ間の遷移を処理します。

・ **ReplayManager** : タイムアタックモード用のリプレイシステムです。プレイ記録を JSON 形式で保存・読み込みし、クリア後に再生可能にします。

・ **GameState** : ゲーム全体の状態を統合管理する構造体です。プレイヤー、カメラ、アイテム、スキル、進行状況、リプレイ、UI などの各サブシステムの状態を保持します。

開発の際に工夫したこと

- ・ SOLID 原則を意識

特に単一責任の原則に基づき、各クラス・関数は 1 つの責任のみを持つことを心がけて設計しています。GameUpdater (更新)、GameRenderer (描画)、InputHandler (入力) などに分離しているため、既存コードをあまり変更せずに新機能を追加できる設計にしています。

・状態管理の分離設計

GameState で各サブシステムの状態（PlayerState、CameraState、ItemState、SkillState 等）を分離しています。状態の変更が他システムに影響しにくく、デバッグとテストが容易になります。

・データ駆動設計によるステージ管理

ステージデータを JSON ファイルで管理しています。そのため、コード変更なしでステージの追加・編集が可能になります。JsonStageLoader で読み込み、StageManager で統合管理しています。

データ構造・データ駆動

- ・ステージ構成の JSON 外部化：ステージごとのプラットフォーム配置、アイテム位置、ギミック設定を JSON ファイルで定義しています。ゲーム起動時に JsonStageLoader で読み込み、動的に生成しているので、コード変更なしでステージの追加・編集が可能です。

- ・セーブデータの JSON 管理：プレイヤーの進行状況、アンロック状態、タイムアタック記録などを JSON 形式で保存・読み込みをしています。

- ・UI 設定の JSON 外部化：UI 要素の位置、色、サイズなどの設定を ui_config.json で管理してゲーム実行中にファイル変更を検知して自動リロードし、調整を効率化しています。

- ・リプレイデータの JSON 保存：タイムアタックモードのリプレイデータを JSON 形式で保存して、フレーム単位のプレイヤー位置・速度・アイテム収集状態を記録し、再生時に再現しています。

4. 開発プロセス

開発プロセスの概要（タイムライン）

2025.08-09：コア実装・全ステージ作成

2025.11：リファクタリング・一部機能追加・提出準備

開発開始時、夏季休暇だったため週 50 時間以上を目処に作業を行っていました。

開発体制・担当範囲

開発体制：個人開発

担当範囲：企画、設計、実装、デバッグなど全ての工程を担当しました。（Github にコラボレーターがいますがそちらは WindowsPC を借りて、自分で作業をしたもの

になります。)

開発における課題とその解決

開発中に直面した課題とその解決について、課題→解決→効果→学びという4つの観点から紹介します。

課題①：ビルドと再起動が必要で調整効率が悪い

問題：ステージやUIを調整するたびにコードを書き換え、ビルド&ゲーム再起動を行っていました。その原因として、ステージ配置やUI情報をハードコードしていたことがあり、細かい調整に時間がかかって試行回数が稼げない状態でした。

解決：ステージ配置・UI情報などをJSONで定義するデータ駆動構成に変更しました。また、ゲーム起動中にJSONファイルの更新を監視し、保存されたタイミングでホットリロードして即座に反映する仕組みを実装しました。

- ・ステージ：ステージ管理用JSONを編集 → 保存 → 起動中のゲーム内ステージに即反映

- ・UI：UI座標・サイズ等をJSONから読み込み → 保存 → 起動中のUIに即反映

効果：ステージの足場配置やUIの位置調整を再起動なしで連続して試せるようになり、細かいチューニングにかかる時間を大幅に削減できました。その結果、ステージ・UIの試行回数が増え、全体の完成度向上にもつながりました。

学び：ゲーム開発では、コンテンツをハードコードせずデータ駆動にし、編集→確認サイクルを短くすることが生産性を大きく上げると実感しました。将来的にデザイナーやレベルデザイナーと協業する際にも、外部データで制御できる構成は有効だと感じました。

課題②：リプレイ再生を滑らかにするとデータ量が増えてしまう

問題：タイムアタックモードでリプレイ機能を実装する際、毎フレーム記録するとデータ量が増えてしまい、一方記録間隔を大きくすると再生時にカクついてしまうという問題が発生しました。例えば60FPSで記録すると30秒のプレイで約1,800フ

レームとなり、ファイルサイズが大きくなります。一方、0.1 秒間隔（10FPS）で記録すると容量は抑えられますが、そのまま再生すると動きが不自然に見えます。

解決： 解決のため、プレイヤー位置などのリプレイ情報を線形補間するようにしました。記録は 0.1 秒間隔で行い、再生時に記録した 2 つのデータ間の位置を補間します。例えば記録時刻が 0.1f、0.2f と離れていて現在時刻が 0.15f だった場合、補間係数 $t = 0.5$ を計算し、 t と `glm::mix` を用いて位置を線形補間します。しかしアイテム取得状態は補間することができないため、離散値として 2 つのデータの中間点で切り替えるようにしています。（0.14f であれば 0.1f 時点での取得状態を、0.16f であれば 0.2f 時点での取得状態を反映させています。）また、巻き戻し・早送りにも対応し、再生速度を変更しても同じ補間アルゴリズムで動作します。

効果： データ量を毎フレーム記録していた場合と比較して約 6 分の 1 程度に削減しつつ、滑らかに再生できるようになりました。実際にユーザーに遊んでもらったことで、補間をしてもユーザーは違和感を覚えないことも確認できました。

学び： データ量と品質のバランスを取る設計の重要性を実感しました。また、連続値と離散値を適切に扱うことで、補間の精度と実装の簡潔さを両立できることを学びました。

課題③：タイムアタック記録を共有しにくい、モチベーションを保ちにくい

問題： タイムアタックモードを実装したものの、ローカルでの自己記録更新だけでは、何度も同じステージをプレイする動機が弱く、モチベーションを保てなかったり、ゲーム自体が単調になりがちでした。また、上位プレイヤーのプレイスタイルを学ぶ機会もなく、技術向上のきっかけが限られていました。

解決： オンラインランキングシステムを実装しました。`libcurl` を使用して HTTP 通信を行い、バックエンド API と非同期でデータを送受信させています。具体的には、ステージクリア時にタイム記録とリプレイデータを自動送信し、ステージ選択画面のランキングボードから他プレイヤーの記録を閲覧できるようにしました。また、データベースからリプレイを取得することで、他のプレイヤーのリプレイを再生できるようにしました。

効果： オンラインランキング機能により、プレイヤー間の競争が生まれ、何度もタイムアタックで遊びたくなる導線を作ることができました。その結果、自分の順位

を確認し、上位プレイヤーのタイムを目標にすることで、継続的なプレイの動機が生まれました。また、リプレイ機能により、上位プレイヤーのプレイスタイルを学習でき、プレイ技術の向上にもつながりました。結果として、タイムアタックモードが単なる自己記録更新から、コミュニティ全体での競争へと発展し、ゲームの単調さが解消されました。

学び： ゲームのリプレイ性を高めるには、単なる機能追加だけでなく、プレイヤー間の競争や学習機会を提供することが重要だと実感しました。また、サーバーとの通信中にゲームが止まらないようにするために、非同期通信を行う重要性を学びました。

5. まとめ

本作品で達成できたこと

- ・ゲーム世界の速度を切り替えながら進む空中アクションというコンセプトを、ノーマル／タイムアタック／シークレットスターといった複数モードに落とし込み、PC 向けのプレイアブルなタイトルとして完成させました。
- ・GLFW や OpenGL を用いて自作の入力、描画などのシステムを構築し、ゲームループ・ステージ／UI・リプレイ・オンラインランキングなど、ゲームプレイ全体を一人で設計・実装できました。
- ・UI／ステージの JSON データ化とホットリロードといった仕組みを整えることで、個人開発でも効率よく開発できる環境を構築できました。

本作品を開発中に学んだこと

- ・コンテンツをハードコードせず JSON などの外部データで管理し、ホットリロード可能にすることで、ステージや UI の調整サイクルが大幅に短縮され、ゲームの完成度向上に直結することを実感しました。
- ・リプレイ機能の実装を通じて、補間を用いたデータ量と再生品質のバランスを取る設計の重要性を理解しました。
- ・イージーモードやシークレットスターモードの設計を通じて、アクションが苦手なプレイヤーから上級者まで、幅広い層が楽しめる難易度設計の大切さを学びました。

今後の展望

- ・ステージ数やギミックのバリエーションを拡充し、特に演出を強化することで、

ゲームとしての体験をさらに厚くしていきたいです。

- ・レベルエディタなど開発用ツールを整備し、JSON+ホットリロードの仕組みを発展させることで、将来的にはレベルデザイナーなど他職種とも協業しやすい環境を目指したいです。